

Improved Dynamic Programming Algorithms for Bandwidth Minimization and the MinCut Linear Arrangement Problem*

EITAN M. GURARI[†]

*Computer and Information Science Department, Ohio State University,
Columbus, Ohio 43210*

AND

IVAN HAL SUDBOROUGH[‡]

*Department of Electrical Engineering and Computer Science, Northwestern University,
Evanston, Illinois 60201*

Received March 19, 1982

The dynamic programming algorithm of J. B. Saxe (*SIAM J. Algebraic Discrete Methods* (1980)) for the bandwidth minimization problem is improved. It is shown that, for all $k > 1$, $\text{BANDWIDTH}(k)$ can be solved in $O(n^k)$ steps and simultaneous $O(n^k)$ space, where n is the number of vertices in the graph, and that each such problem is in $\text{NSPACE}(\log n)$. The same improved dynamic programming algorithm approach works to show that the $\text{MINCUT LINEAR ARRANGEMENT}$ problem restricted to the fixed value k , denoted by $\text{MINCUT}(k)$, is solvable in $O(n^k)$ steps and simultaneous $O(n^k)$ space and is in the class $\text{NSPACE}(\log n)$.

© 1984 Academic Press, Inc.

1. INTRODUCTION

Let $G = (V, E)$ be a finite undirected graph. A one-to-one function $L: V \rightarrow \{1, \dots, |V|\}$ is a *linear layout* or simply a *layout* of G . (The term "layout," used instead of the more traditional term "vertex labeling," is motivated by applications of the $\text{BANDWIDTH MINIMIZATION}$ and $\text{MINCUT LINEAR ARRANGEMENT}$ problems in VLSI circuit design.)

*Portions of this work appeared previously in the proceedings of the 18th and 19th Annual Allerton Conference on Communication, Control, and Computing [4, 9].

[†]Supported in part by NSF Grant No. MCS 80-21124.

[‡]Supported in part by NSF Grant No. MCS 79-08919 and in part by NSF Grant No. 81-09280.

Let k be a positive integer. G has bandwidth k under the layout L , denoted by bandwidth $(G, L) = k$, if $|L(x) - L(y)| \leq k$, for all edges $\{x, y\}$ in G . G has cutwidth k under the layout L , denoted by cutwidth $(G, L) = k$, if $\max\{|E(i)| \mid 1 \leq i < |V|\} \leq k$, where $E(i) = \{\{x, y\} \in E \mid L(x) \leq i < L(y)\}$.

The BANDWIDTH MINIMIZATION PROBLEM, deciding for a given graph G and a positive integer k whether there exists a layout L such that bandwidth $(G, L) = k$, has long been of interest in sparse matrix computations [6]. It is known to be NP-complete [10]. Furthermore, it is known to remain NP-complete when the input graphs G are restricted to free trees of degree at most three [2].

Similarly, the MINIMUM CUT LINEAR ARRANGEMENT PROBLEM, deciding for a given graph G and positive integer k whether there exists a layout L such that cutwidth $(G, L) = k$, has several interesting applications. For instance, Lengauer [5] points out that the cutwidth of a graph times the size of the graph gives a measure of the area needed to represent the graph in VLSI applications in which the vertices (or active circuit elements) are laid out in a row. The MINCUT problem is also involved in the optimal placement of gates in the so-called Weinberger array approach to VLSI [14, 15]. It is known that this MINCUT problem is NP-complete [3, 13]; in fact, it remains NP-complete when restricted to graphs with maximum vertex degree three [7]. However, in contrast with the BANDWIDTH MINIMIZATION PROBLEM, the problem MINCUT when restricted to trees can be done in polynomial time [1, 16].

In this paper we consider the family of problems obtained by restricting the input integer in these two general problems to various fixed values. For the fixed value k the resulting problems will be denoted by BANDWIDTH(k) and MINCUT(k), respectively. It is known that the problem BANDWIDTH(2) can be solved in linear time [2]. Similarly, a recent result indicates that MINCUT(2) can be solved in linear time [8]. Also, Saxe has described a dynamic programming algorithm for each problem BANDWIDTH(k) that works in $O(n^{k+1})$ steps and $O(n^{k+1})$ space [12]. Our results improve on the bounds given by Saxe and show how to extend the basic method to each MINCUT(k) problem. We show that, for each fixed value k , the problems BANDWIDTH(k) and MINCUT(k) can be solved in $O(n^k)$ steps and $O(n^k)$ space. Furthermore, for each fixed value k , the problems BANDWIDTH(k) and MINCUT(k) are shown to be in NSPACE($\log n$).

2. IMPROVED ALGORITHMS FOR BANDWIDTH(k) AND MINCUT(k)

As we stated earlier, our algorithms are improvements on the basic dynamic programming algorithms described by Saxe [12]. We review some

of the definitions and concepts from Saxe's paper for completeness. We assume in the described algorithms that the input graph is connected, since a graph G has bandwidth (cutwidth) k if and only if every connected component of G has bandwidth (cutwidth) k . Also, we observe that a graph G can only have bandwidth (cutwidth) k if each of its vertices has degree at most $2k$. It is known that a graph can be partitioned into its connected components by depth first search in $O(\max(n, e))$ steps, where n is the number of vertices and e is the number of edges. This time bound is equivalent to an $O(n)$ time bound when the graph has a fixed vertex degree, as all graphs with bandwidth (cutwidth) k must have.

DEFINITION 2.1. A *partial layout* of the graph $G = (V, E)$ is a one-to-one function L from some subset V' of V to the set $\{1, \dots, |V'|\}$. The *bandwidth of the partial layout* L , denoted by $\text{bandwidth}(L)$, is $\max\{|L(x) - L(y)| \mid \{x, y\} \text{ is an edge of } G \text{ and } x \text{ and } y \text{ are in the domain of } L\}$. The *cutwidth of the partial layout* L , denoted by $\text{cutwidth}(L)$, is $\max\{|E(i)| \mid 1 \leq i < |V'|\}$, where $E(i) = \{\{x, y\} \text{ in } E \mid L(x) \leq i \text{ and either } L(y) > i \text{ or } L(y) \text{ is undefined}\}$. If $e = \{x, y\}$ is an edge, x is in the domain of L , and y is not in the domain of L , then e is called a *dangling edge* (from L). The set of all dangling edges is denoted by $\text{dangling}(L)$. A vertex x in $\text{domain}(L)$ is *active* (in L) if it is incident to a dangling edge. The set of active vertices is denoted by $\text{active}(L)$.

Let L be a partial layout which maps m of the vertices of G onto the set $\{1, \dots, m\}$. The *active region* of L , denoted by $\text{region}(L)$, is the sequence $(L^{-1}(m-p), L^{-1}(m-p+1), \dots, L^{-1}(m))$, where $p = \min\{i \mid \text{active}(L) \subseteq \{L^{-1}(m-i), L^{-1}(m-i+1), \dots, L^{-1}(m)\}\}$.

Let $p = (r, d)$ be a pair consisting of a sequence of $r = (v_1, v_2, \dots, v_k)$ of vertices and a set d of edges. The pair p represents a class of partial layouts in a natural way. That is, the class of partial layouts represented by the pair p , denoted by $\langle p \rangle$, is the set of all partial layouts L such that $\text{region}(L) = r$ and $\text{dangling}(L) = d$. The collection of all such classes forms a partition of the family of partial layouts of G and, hence, induces an equivalence relation on partial layouts. This equivalence relation asserts that partial layouts L_1 and L_2 are equivalent if $(\text{region}(L_1), \text{dangling}(L_1)) = (\text{region}(L_2), \text{dangling}(L_2))$. We want to work with a refinement of this equivalence relation. L_1 is *bandwidth equivalent* to L_2 if (a) $\text{bandwidth}(L_1) = \text{bandwidth}(L_2)$ and (b) $(\text{region}(L_1), \text{dangling}(L_1)) = (\text{region}(L_2), \text{dangling}(L_2))$. We show that, if L_1 is bandwidth equivalent to L_2 , then the domain of the two partial layouts, L_1 and L_2 , must be the same and L_1 can be extended to a bandwidth k layout of the entire graph G if and only if L_2 can be extended to a bandwidth k layout of the entire graph G .

LEMMA 2.1. *Let L_1 and L_2 be bandwidth equivalent partial layouts of a graph $G = (V, E)$. Then, (1) $\text{domain}(L_1) = \text{domain}(L_2)$ and (2) for any $k \geq 1$, either both L_1 and L_2 can be extended to a bandwidth k layout of the entire graph G or neither can be so extended.*

Proof. That $\text{domain}(L_1) = \text{domain}(L_2)$ follows from the fact that the vertices in the domain of any partial layout of a connected graph are precisely those reachable by a path from the active vertices in which no dangling edge is traversed. In other words, the partition of the graph G into the two sets of vertices, $\text{domain}(L)$ and $V - \text{domain}(L)$, forms a cut of G in which only the dangling edges connect vertices in opposite sets. Also, as noted by Saxe [12], any layout of the remaining vertices that extends L_1 to a bandwidth k layout of the entire graph G will also extend L_2 to a bandwidth k layout of the entire graph.

The algorithm to determine if a graph G has bandwidth k considers classes of partial layouts L , as represented by pairs $p = (r, d)$, where $r = \text{region}(L)$ and $d = \text{dangling}(L)$. The algorithm begins by considering the partial layout L such that $\text{domain}(L) = \emptyset$, which is represented by the pair $(\emptyset, \emptyset)$. For each class of partial layouts considered, the algorithm tries all possible vertices which possibly can be used to extend the layouts to bandwidth k layouts of the entire graph G . Clearly, a partial layout L cannot be extended to a bandwidth k layout of the entire graph G if $\text{region}(L)$ consists of more than k vertices. For if $\text{region}(L)$ consists of more than k vertices, then at least one of the dangling edges must connect vertices mapped to integers more than k apart. In fact, several other properties are required of any partial layout L , which can be extended to a bandwidth k layout of the entire graph G . Let L be a partial layout which maps m vertices of G onto the set $\{1, \dots, m\}$. L can be extended to a bandwidth k layout of the entire graph G only if

- (1) $\text{bandwidth}(L) \leq k$,
- (2) $\text{active}(L) \subseteq \{L^{-1}(m - k + 2), L^{-1}(m - k + 1), \dots, L^{-1}(m)\}$,
and
- (3) the vertex $L^{-1}(m - j + 1)$, for all j ($1 < j < k$), is incident to at most $k - j + 1$ dangling edges.

Condition (1) is clearly necessary and condition (2) is a restatement of the fact that $\text{region}(L)$ must consist of at most k vertices. We note that condition (3) asserts that there must be at most one dangling edge incident to the vertex $L^{-1}(m - k + 1)$. This follows from the fact that any dangling edge that is incident to $L^{-1}(m - k + 1)$ must connect to the vertex assigned in any extension of L to the next available integer; otherwise, the bandwidth k requirement would be violated. So, there can only be one

dangling edge incident to the vertex $L^{-1}(m - k + 1)$. The other parts of condition (3) are established in a similar manner.

The algorithm works in such a way that conditions (1) and (2) are guaranteed. Only condition (3) needs to be checked explicitly. We shall say that a pair $p = (r, d)$, which represents a class of bandwidth equivalent partial layouts, is *bandwidth- k -plausible*, where $r = (v_1, v_2, \dots, v_i)$, if there are at most $k - i + j$ edges in the set d incident to v_j , for all j ($1 \leq j \leq i$). In particular, when $r = (v_1, v_2, \dots, v_k)$, this means that the vertex v_1 can have at most one dangling edge. The algorithm separates the pairs it considers into two cases: (a) the active region r consists of a sequence of k vertices and (b) the active region r consists of fewer than k vertices. In case (a) the vertex assigned to the next available integer must be the one at the other end of the dangling edge incident to the first vertex in the region. Thus, in case (a), there is a unique next pair and this pair can be found easily. In case (b) there may be several vertices that can be assigned to the next available integer and, hence, several new equivalence classes will need to be considered. The algorithm will try all of them.

The algorithm uses two data structures:

(1) a (fifo) queue Q whose elements are bandwidth- k -plausible pairs $p = (r, d)$, and

(2) an array T which contains one element for each bandwidth- k -plausible pair $p = (r, d)$. The element $T\langle p \rangle$ consists of a boolean value such that $T\langle p \rangle$ is true if and only if the pair $p = (r, d)$ has already been considered during the current run of the algorithm.

At the start of the algorithm T is initialized so that $T\langle p \rangle$ is false for all pairs p . We have dispensed with the explicit listing of the unassigned vertices in the data structures, since the unassigned vertices can be computed from the pair p in a number of steps bounded by some constant multiple of the number of unassigned vertices. This computation is done by the subroutine Unassigned shown in Fig. 2.1. Unassigned(r, d) computes the set of all vertices not in the domain of any partial layout in the equivalence class $\langle p \rangle$, where $p = (r, d)$. Not explicitly listing the unassigned vertices in the array T saves considerable space and does not add to the asymptotic running time performance of the algorithm. The savings in time that is achieved by our algorithm over that given by Saxe is due to the fact that we are considering fewer equivalence classes and also that for several of these equivalence classes we need only a few steps to decide which vertex must be assigned next.

The procedure Unassigned works as follows. Initially the algorithm is given a pair $p = (r, d)$, where r is an active region and d is a set of dangling edges. The unassigned vertices are exactly those reachable from the

active vertices by a path that begins with a dangling edge and never again traverses a dangling edge. Let x be the unassigned vertex at the end of one of the dangling edges. The procedure Unassigned computes the pair $p' = (r', d')$, where r' is the sequence in which x is added to the end of the sequence r and d' is the set of dangling edges in this new partial layout. The procedure Unassigned can then proceed to compute additional unassigned vertices by the same process using the new pair $p' = (r', d')$.

The procedure Unassigned uses three data structures:

- (1) a queue Q' whose elements are edges in G ,
- (2) a boolean array DG which contains one element for each edge in G ; the element $DG\langle e \rangle$ is true if and only if e is a dangling edge, i.e., e is in the set d of the current pair $p = (r, d)$,
- (3) a boolean array A which contains one element for each vertex of G ; the element $A(v)$ is true if and only if v is an active vertex of any partial layout in the current equivalence class $\langle p \rangle$ or v has been an active vertex at some point since the initial call to the procedure.

The algorithm also uses a procedure called Update which computes, for a given pair $p = (r, d)$ and a given vertex s , the pair that describes the

```

Unassigned( $R, D$ )
begin
 $V \leftarrow \emptyset$ ;
 $Q' \leftarrow \emptyset$ ;
if  $D = \emptyset$  then return  $\{x | x \text{ is a vertex of } G\}$ 
for all edges  $e$  in  $D$  do
  begin  $DG(e) \leftarrow \text{true}$ ; add  $e$  to  $Q'$  end
for all  $v$  in  $R$  do
   $A(v) \leftarrow \text{true}$ ;
while  $Q'$  is not empty do
  begin
    delete an edge  $e = \{x, y\}$  from  $Q'$ ;
    if  $A(x) = \text{false}$  or  $A(y) = \text{false}$  then
      begin
        if  $A(x) = \text{false}$  then  $z \leftarrow x$  else  $z \leftarrow y$ ;
         $A(z) \leftarrow \text{true}$ ;
         $V \leftarrow V \cup \{z\}$ ;
        for each edge  $e$  incident to  $z$  do
          if  $DG(e) = \text{false}$  then
            begin add  $e$  to  $Q'$ ;  $DG(e) \leftarrow \text{true}$ 
            end
          end
      end
  end
return  $V$ 
end

```

FIG. 2.1. The procedure Unassigned.

equivalence class of all partial layouts that result from assigning the vertex s to the next available integer. The procedure Update needs to compute the active region after the vertex s is assigned and it needs to compute the new set of dangling edges. The procedure Update is shown in Fig. 2.2

We note that the procedure Update requires some constant c number of steps, where c depends only upon the value of k and not on the number of vertices in G . That is, for any pair $p = (r, d)$, there are at most k vertices in the active region r and there are at most $2k^2$ edges in d , since each vertex in r can be incident to at most $2k$ dangling edges. Update takes a number of steps proportional to the number of edges in d and the number of vertices in r each time it is called.

The algorithm for determining if a graph G has bandwidth k is shown in Fig. 2.3. We note that

(1) if $p = (r, d)$ is a pair such that r is a region of k vertices, then there is only one next pair $p' = (r', d')$ and the algorithm takes some constant number of steps to obtain this new pair and, if appropriate, add it to the queue Q ;

(2) if $p = (r, d)$ is a pair such that r is a region of fewer than k vertices, then there are at most $O(n)$ next pairs to consider, where n is the number of vertices, and the algorithm spends some constant number of steps for each of these new pairs;

(3) there are at most $O(n^k)$ pairs $p = (r, d)$ such that r consists of k vertices; and

(4) there are at most $O(n^{k-1})$ pairs $p = (r, d)$ such that r consists of fewer than k vertices.

That the algorithm requires at most $O(n^k)$ steps follows from the above observations, since $O(n^k) \cdot O(1) + O(n^{k-1}) \cdot O(n) = O(n^k)$. Furthermore, the algorithm needs at most $O(n^k)$ space to store the array T , the queue Q ,

```

Update (  $p, s$  )
/let  $p = (R, D)$ , where  $R = (v_1, v_2, \dots, v_i)$  and  $D = \{e_1, e_2, \dots, e_j\}$ /
begin
  for each edge  $e$  in  $D$  do
    if  $e$  is incident to  $s$  then delete  $e$  from  $D$ ;
   $k \leftarrow 1$ ;
  while  $v_k$  is not incident to an edge in  $D$  do
     $k \leftarrow k + 1$ ;
  for each edge  $e$  incident to  $s$  do
    if  $e$  is not incident to a vertex in  $(v_1, v_2, \dots, v_i)$ 
      then add  $e$  to  $D$ ;
  return the new pair  $p' = ((v_k, v_{k+1}, \dots, v_i, s), D)$ 
end

```

FIG. 2.2. The procedure Update.

the arrays A and DG , and the queue Q' . (This is only true, of course, if we consider that each pair $p = (r, d)$ can be represented using a constant amount of space. In fact, it might be more appropriate to consider that each pair $p = (r, d)$ requires $O(\log n)$ space; in which case, the algorithm requires $O(n^k \cdot \log n)$ space.)

THEOREM 2.1. *For all $k \geq 1$, $BANDWIDTH(k)$ can be solved by a deterministic algorithm in $O(n^k)$ steps and in $O(n^k)$ space.*

It should, perhaps, be noted that the algorithm Bandwidth shown in Fig. 2.3 can be speeded up somewhat in practice. That is, we can separate into several additional cases the current pair $p = (r, d)$, depending upon the structure of the dangling edges. For example, suppose the algorithm is determining if a graph G has bandwidth k and suppose that the current pair $p = (r, d)$ is such that the region $r = (v_2, v_3, \dots, v_k)$ consists of $k - 1$ vertices (as indicated). Then, if there are two dangling edges incident to v_2 , or three dangling edges incident to v_3 , or four dangling edges incident to $v_4, \dots$, or k dangling edges incident to v_k , then the vertex to be assigned to the next available integer is easily obtained, since it must be a vertex at the end of one of the dangling edges. In other words, an algorithm need not try all possible choices for the next vertex as the algorithm described in Fig. 2.3 does. We have written the algorithm in this way for ease of description,

```

Bandwidth( $G, k$ )
begin add the pair  $p = (\emptyset, \emptyset)$  to  $Q$ ;
while  $Q \neq \emptyset$  do
  begin delete a pair  $p = (r, d)$  from  $Q$ ;
    if  $r$  is a sequence  $\{v_1, v_2, \dots, v_k\}$  of  $k$  vertices
    then begin
      let  $e = \{v_1, s\}$  be the dangling edge incident to  $v_1$ ;
       $p' \leftarrow \text{Update}(p, s)$ ; /let  $p' = (r', d')$ /
      if  $d' = \emptyset$  then stop and answer " $G$  has bandwidth  $k$ "
      if  $p'$  is bandwidth- $k$ -plausible and  $T\langle p' \rangle = \text{false}$ 
      then begin  $T\langle p' \rangle \leftarrow \text{true}$ ; add  $p'$  to  $Q$  end
    end
  else begin
     $V \leftarrow \text{Unassigned}(r, d)$ ;
    for all  $s$  in  $V$  do begin
       $p' \leftarrow \text{Update}(p, s)$ ; /let  $p' = (r', d')$ /
      if  $d' = \emptyset$  then stop and answer " $G$  has bandwidth  $k$ "
      if  $p'$  is bandwidth- $k$ -plausible and  $T\langle p' \rangle = \text{false}$ 
      then begin  $T\langle p' \rangle \leftarrow \text{true}$ ; add  $p'$  to  $Q$  end
    end
  end
end
stop and answer " $G$  has bandwidth larger than  $k$ "
end

```

FIG. 2.3. An algorithm for determining if a graph G has bandwidth k .

since the improved running time does not justify a better asymptotic upper bound. Other observations also can lead to improvements in practice. For example, if there are k dangling edges and these edges are incident to k distinct vertices, then the next vertex must be chosen to be one of those at the end of a dangling edge. Clearly, such observations can eliminate several unnecessary pairs from being considered in the running of our dynamic programming algorithm.

We observe that a similar algorithm works for the problem MINCUT(k). In the case of MINCUT(k) we do not need to keep track of the whole region of active vertices. We need only remember the set of active vertices and, of course, the set of dangling edges. Let L_1 and L_2 be two partial layouts. L_1 and L_2 are cutwidth equivalent if (a) $(\text{active}(L_1), \text{dangling}(L_1)) = (\text{active}(L_2), \text{dangling}(L_2))$ and (c) $\text{cutwidth}(L_1) = \text{cutwidth}(L_2)$. It is obvious that the proof of Lemma 2.1 also suffices to prove the following:

LEMMA 2.2. *Let L_1 and L_2 be cutwidth equivalent partial layouts of a graph $G = (V, E)$. Then (1) $\text{domain}(L_1) = \text{domain}(L_2)$ and (2) for any $k \geq 1$, either both L_1 and L_2 can be extended to a cutwidth k layout of the entire graph G or neither can be so extended.*

The algorithm to determine if a graph G has cutwidth k considers classes of partial layouts L , as represented by a pair $p = (a, d)$, where $a = \text{active}(L)$ and $d = \text{dangling}(L)$. The algorithm begins by considering the partial layout L such that $\text{domain}(L) = \emptyset$, which is represented by the pair $(\emptyset, \emptyset)$. For each class of partial layouts considered the algorithm tries all possible vertices which can possibly be used to extend the partial layouts to a cutwidth k layout of the entire graph G . Clearly it is necessary for a partial layout L to satisfy the condition $\text{cutwidth}(L) \leq k$, if it is to be possible to extend L to a cutwidth k layout of the entire graph. We shall say that a partial layout L is *cutwidth- k -plausible*, if $\text{cutwidth}(L) \leq k$. In particular, this means that there can be at most k dangling edges in the partial layout L . If there are precisely k edges dangling in the partial layout L , then there are k vertices that can possibly be assigned to the next available integer. These are the k unassigned vertices at the ends of the dangling edges. For if some other vertex were assigned to the next available integer, then the cutwidth k condition would be violated, since the assigned vertex must be incident to at least one edge by the assumption that the graph G is connected. On the other hand, if $\text{dangling}(L)$ contains fewer than k edges, then conceivably any vertex could be chosen to assign to the next available integer. Thus, we shall separate the pairs $p = (a, d)$ in which there are k dangling edges from the pairs in which there are fewer than k dangling edges.

At the start of the algorithm T is initialized so that $T\langle p \rangle$ is false for all pairs p . The procedure Cutwidth is shown in Fig. 2.4; it uses two data

structures:

(1) a queue Q whose elements are cutwidth- k -plausible pairs $p = (r, d)$ and

(2) an array T which contains one entry for each cutwidth- k -plausible pair $p = (a, d)$. The element $T\langle p \rangle$ consists of a boolean value such that $T\langle p \rangle$ is true if and only if the pair $p = (a, d)$ has already been considered during the current run of the algorithm.

The algorithm Cutwidth uses two procedures named Unassigned and Successor. Unassigned has been described in Fig. 2.1. Successor is described in Fig. 2.5. It is similar to the procedure Update (described in Fig. 2.2); however, Successor computes the set of active vertices in the new partial layout without regard to the order of appearance of these vertices.

It can be seen that the algorithm Cutwidth operates in $O(n^k)$ steps and needs at most $O(n^k)$ units of memory. This follows from essentially the same analysis as given before for the algorithm Bandwidth. In this case, if the pair $p = (a, d)$ has k dangling edges, then the algorithm Cutwidth considers at most k successor pairs $p' = (a', d')$. It takes some constant number of steps to obtain each successor by the procedure Successor and

```

Cutwidth( $G, k$ )
begin add the pair  $p = (\emptyset, \emptyset)$  to  $Q$ ;
while  $Q$  is not empty do
  begin delete a pair  $p = (a, d)$  from  $Q$ ;
    if  $d$  is a set of  $k$  edges then
      for all unassigned vertices  $x$  incident to an edge in  $d$  do
        begin
           $p' \leftarrow \text{Successor}(p, x)$ ; / let  $p' = (a', d')$ /
          if  $d' = \emptyset$  then stop and answer " $G$  has cutwidth  $k$ ";
          if  $p'$  is cutwidth- $k$ -plausible and  $T\langle p' \rangle = \text{false}$ 
            then begin  $T\langle p' \rangle \leftarrow \text{true}$ ; add  $p'$  to  $Q$  end
          end
        else begin
           $V \leftarrow \text{Unassigned}(a, d)$ ;
          for all  $s$  in  $V$  do begin
             $p' \leftarrow \text{Successor}(p, s)$ ; / let  $p' = (a', d')$ /
            if  $d' = \emptyset$  then stop and answer " $G$  has cutwidth  $k$ ";
            if  $p'$  is cutwidth- $k$ -plausible and  $T\langle p' \rangle = \text{false}$ 
              then begin  $T\langle p' \rangle \leftarrow \text{true}$ ; add  $p'$  to  $Q$  end
            end
          end
        end
      stop and answer " $G$  has cutwidth larger than  $k$ "
    end
  end
end

```

FIG. 2.4. An algorithm to determine if a graph G has cutwidth k .

some additional constant number of steps to decide whether to add this new pair to the queue Q . There are $O(n^k)$ distinct pairs $p = (a, d)$ in which d consists of k dangling edges. Therefore, at most $O(n^k)$ steps are spent in this part of the algorithm.

If the pair $p = (a, d)$ has $k - 1$ or fewer dangling edges, then there are at most $O(n)$ vertices that can be assigned to the next integer. The procedure Cutwidth needs some constant number of steps to obtain each successor p' and some additional constant number of steps to decide whether to add p' to the queue Q . Since there are $O(n^{k-1})$ pairs $p = (a, d)$ in which d consists of $k - 1$ or fewer dangling edges, the algorithm Cutwidth spends at most $O(n^{k-1}) \cdot O(n) = O(n^k)$ steps in this part of the algorithm. Thus, an upper bound on the number of steps used by the whole algorithm is $O(n^k)$, since each time the while loop is executed one or the other of these parts of the algorithm must be executed. So, Cutwidth(G, k) decides if a graph G has cutwidth k in at most $O(n^k)$ steps.

THEOREM 2.2. *For all $k \geq 1$, MINCUT(k) can be solved by a deterministic algorithm in $O(n^k)$ steps and in $O(n^k)$ space.*

We observe that the algorithms presented for Bandwidth(k) and Cutwidth(k) can be augmented to provide a layout of the graph with bandwidth k or cutwidth k , respectively. This modification has been described by Saxe [12]. That is, we add to the table T an additional component, called $T\langle p \rangle \cdot \text{predecessor}$, which indicates for any given pair $p = (r, d)$ which pair $p' = (r', d')$ was current when p was added to the queue. Then whenever we encounter a pair $p = (r, d)$ in which d is empty, we can trace back through these components to reconstruct an appropriate layout.

```

Successor( $p, x$ )
/ let  $p = (A, D)$  /
begin
  for each edge  $e$  in  $D$  do
    if  $e$  is incident to  $x$  then delete  $e$  from  $D$ ;
  for each edge  $e$  incident to  $x$  do
    if  $e$  is not incident to a vertex in  $A$  then add  $e$  to  $D$ ;
  for each vertex  $v$  in  $A$  do
    if  $v$  is not incident to any edge in  $D$  then delete  $v$  from  $A$ ;
  if  $x$  is incident to an edge in  $D$  then add  $x$  to  $A$ ;
  return the new pair  $p' = (A, D)$ 
end

```

FIG. 2.5. Successor(p, x) yields the next pair $p' = (a, d)$, where a is the new set of active vertices and d is the new set of dangling edges, when the vertex x is used to extend the partial layouts in the class $\langle p \rangle$.

3. LOG SPACE ALGORITHMS FOR BANDWIDTH(k) AND MINCUT(k)

For each fixed value of k the problems BANDWIDTH(k) and MINCUT(k) are in NSPACE($\log n$). Nondeterministic procedures for these problems are presented in Figs. 3.1 and 3.2, respectively. BAND(G, k) accepts a connected graph G if and only if G has bandwidth k and CUT(G, k) accepts a connected graph G if and only if G has cutwidth k .

It may be necessary to convince the reader that these nondeterministic procedures, BAND and CUT, for a given value k , accept a connected graph G if and only if the graph G has bandwidth or cutwidth k . For instance, it may not be clear that these algorithms, when they reach the statement "stop

```
(nondeterministic) procedure BAND( $G, k$ )
begin
  guess a sequence  $r = (v_1, v_2, \dots, v_k)$  of  $k$  vertices;
  let  $d$  be the set of edges dangling from  $r$ ;
  while  $d$  is not empty do
    begin
      guess a vertex  $x$ ;
      if a dangling edge incident to  $v_1$  is not incident to  $x$ 
        then stop without accepting;
       $(v_1, v_2, \dots, v_k) \leftarrow (v_2, v_3, \dots, v_k, x)$ ;
      let  $d$  be the set of edges dangling from  $(v_1, v_2, \dots, v_k)$ 
    end
  stop and accept
end
```

FIG. 3.1. A nondeterministic log space algorithm for bandwidth (k).

```
(nondeterministic) procedure CUT( $G, k$ )
begin
  guess a set  $A = \{v_1, v_2, \dots, v_k\}$  of  $k$  vertices;
  let  $d$  be the set of edges dangling from vertices in  $A$ ;
  while  $d$  is not empty do
    begin
      guess a vertex  $x$ ;
      if there are more than  $k$  edges in  $d$ 
        then stop without accepting;
      let  $d$  be the new set of dangling edges / i.e., delete any edge from the old set  $d$  that is
        incident to  $x$  and add to the old set  $d$  any edge that is incident to  $x$  but is not
        incident to any vertex in  $A$  /
      let  $A$  be the new set of active vertices
    end
  stop and accept
end
```

FIG. 3.2. A nondeterministic log space algorithm for MINCUT(k).

and accept," have actually verified that an appropriate bandwidth k layout or cutwidth k layout exists. Both algorithms work by guessing a vertex to assign at each iteration of the while loop and they never check that the vertex chosen has not been assigned an integer at some earlier stage. However, the reader should note that if there is an assignment L of integers to vertices (L need not be one-to-one) which satisfies the bandwidth or cutwidth k requirements, then there is also a one-to-one assignment L' of vertices to integers that satisfies the bandwidth k or cutwidth k requirements. For example, an appropriate layout L' can be obtained from such an assignment L by defining $L'(x) = \min\{i | L(i) = x\}$. Notice, for example, that if x and y are adjacent vertices in G and $L'(x) = i$ and $L'(y) = j$, where $i < j$, then it follows that $j \leq i + k$, since otherwise the algorithm BAND would detect a dangling edge from the vertex x when it is leftmost in the region r . In this way the reader can convince himself that if $\text{BAND}(G, k)$ or $\text{CUT}(G, k)$ have an accepting computation, then G has indeed a bandwidth k or cutwidth k layout, respectively.

The other direction is perhaps easier to see. If G has bandwidth or cutwidth k , then $\text{BAND}(G, k)$ or $\text{CUT}(G, k)$ has the possibility of guessing a layout of G that has bandwidth k or cutwidth k , respectively. Since under a sequence of guesses that produces such a layout neither algorithm, BAND or CUT, will ever find any problem with dangling edges, they will eventually complete the layout and, therefore, enter the "stop and accept" statement. So, BAND and CUT are correctly formulated nondeterministic algorithms for the problems $\text{BANDWIDTH}(k)$ and $\text{MINCUT}(k)$, respectively, when the graphs are connected.

The connectivity requirement follows from the fact that both algorithms terminate when the set of dangling edges is empty. If the graph G has several connected components, then the algorithms will terminate after only one of the components is checked for bandwidth k or cutwidth k , respectively. Since there is no known nondeterministic log space procedure to determine if two vertices lie in distinct connected components, the reader might incorrectly surmise that the problems $\text{BANDWIDTH}(k)$ and $\text{MINCUT}(k)$ cannot be done nondeterministically in $\log n$ space for graphs with more than one connected component. However, these problems can be done nondeterministically in $\log n$ space, even for graphs with more than one connected component, by iterating the procedures BAND and CUT. That is, these procedures can be run on the connected component containing each of the n vertices in the graph. The whole graph G has bandwidth k or cutwidth k if and only if each of these connected components has bandwidth k or cutwidth k , respectively.

The algorithms work in $\log n$ space, where n is the number of vertices, since both BAND and CUT need only store the index of a fixed number of vertices and a fixed number of edges in memory. Each such index requires

at most $O(\log n)$ space using binary notation. Thus, any fixed number of such indices can also be stored in $O(\log n)$ memory cells. So, it follows that $\text{BANDWIDTH}(k)$ and $\text{MINCUT}(k)$ are in $\text{NSPACE}(\log n)$, for each fixed value k . It is not apparent that any of these problems can be done deterministically in $O(\log n)$ space, although this is presumably true for sufficiently small values of k . (For example, graphs with cutwidth 2 and cutwidth 3 have been characterized in [8]. Using the first of these characterizations it would seem to be possible to construct a deterministic algorithm for recognizing cutwidth 2 graphs that takes $O(\log n)$ space.)

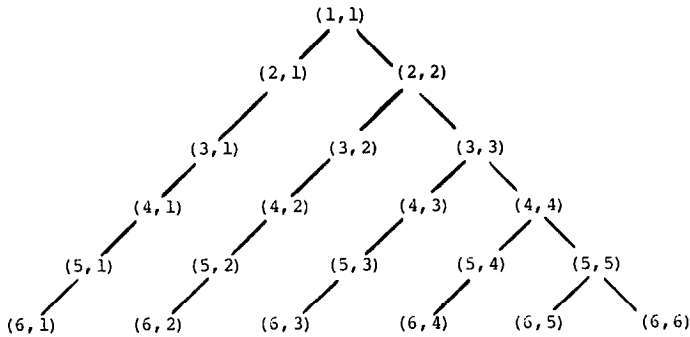
THEOREM 3.1. *For all $k \geq 1$, $\text{BANDWIDTH}(k)$ and $\text{MINCUT}(k)$ are in $\text{NSPACE}(\log n)$.*

A natural question arises: Are any of the problems $\text{BANDWIDTH}(k)$ or $\text{MINCUT}(k)$, for sufficiently large k , log space complete for $\text{NSPACE}(\log n)$? That is, is it possible to transform the graph accessibility problem [11] into one of these bandwidth minimization problems or min cut linear arrangement problems? It is also natural to ask if these problems are related to each other by log space transformations or whether they form an infinite collection of equivalence classes (under log space reductions) between $\text{DSPACE}(\log n)$ and $\text{NSPACE}(\log n)$. While it does not seem difficult to find a log space reduction from $\text{BANDWIDTH}(k)$ to $\text{BANDWIDTH}(k+1)$ and from $\text{MINCUT}(k)$ to $\text{MINCUT}(k+1)$, for each $k \geq 1$, it does not seem to be easy to find a log space reduction from $\text{BANDWIDTH}(k+1)$ to $\text{BANDWIDTH}(k)$ or from $\text{MINCUT}(k+1)$ to $\text{MINCUT}(k)$ for arbitrary values of k .

We note that if L is a layout of a graph G such that $\text{bandwidth}(L) \leq k$, then $\text{cutwidth}(L) \leq k(k+1)/2 = O(k^2)$. This follows from the observation that any cut between successive vertices in a bandwidth k linear layout can only contain edges that connect vertices that are at most k apart. Furthermore, if the graph G has fixed vertex degree, then any linear layout with bandwidth k has cutwidth $O(k)$. So, a bandwidth restriction on a graph implies a corresponding cutwidth restriction. The converse, however, is false. This is shown below.

Consider the family of trees $\{T(k) \mid k \geq 1\}$, where $T(k) = (V(k), E(k))$, with $V(k) = \{(i, j) \mid 1 \leq i \leq k \text{ and } 1 \leq j \leq i\}$ and $E(k) = \{(i, j), (i+1, j)\} \mid 1 \leq i \leq k \text{ and } 1 \leq j \leq i\} \cup \{(i, i), (i+1, i+1)\} \mid 1 \leq i \leq k\}$. The tree $T(6)$ is described in Fig. 3.3.

It is straightforward to show that, for all $k \geq 1$, the tree $T(k)$ has a cutwidth 2 layout. However, as is well known, the bandwidth of a graph must be at least as large as the size of the graph minus one divided by the diameter of the graph, where the diameter of a graph G is maximum $\{d(x, y) \mid x, y \text{ in } G\}$ and $d(x, y)$ denotes the length of the shortest path

FIG. 3.3. The tree $T(6)$.

from the vertex x to the vertex y . This implies that the bandwidth of $T(k)$, as k approaches infinity, approaches infinity. That is, $T(k)$ contains $k(k+1)/2$ vertices and the diameter of $T(k)$ is $2(k-1)$. Therefore, the bandwidth of $T(k)$ is $O(k)$.

COMMENTS

It has recently been shown that the $\text{MINCUT}(k)$ problem can be solved deterministically in $O(n^{k-1})$ steps [8] by an improved version of the dynamic programming algorithm given here. An earlier form of the $\text{Bandwidth}(k)$ algorithm used k -head deterministic pushdown automata [9]. In fact the algorithm Band , described in Fig. 3.1, can be described as a k -head nondeterministic finite automaton algorithm. This would yield an $O(n^k)$ bound for $\text{Bandwidth}(k)$, where n is the length of the input. These initial considerations, moreover, led us to the results of this paper and thereby suggest once again that automata theory can be useful.

ACKNOWLEDGMENTS

We would like to express our appreciation to Jim Saxe and Burkhard Monien for helpful conversations during the early stages of our work.

REFERENCES

1. M. J. CHUNG, F. MAKEDON, I. H. SUDBOROUGH, AND J. TURNER, Polynomial algorithms for the min-cut linear arrangement problem on degree restricted trees, in "Proceedings, 23rd Annual IEEE Foundations of Computer Science Symposium, 1982," pp. 262-271.
2. M. R. GAREY, R. L. GRAHAM, D. S. JOHNSON, AND D. E. KNUTH, Complexity results for bandwidth minimization, *SIAM J. Appl. Math.* **34** (1978), 477-495.

3. F. GAVRIL, Some NP-complete problems on graphs, in *Proceedings 11th Annual Conference on Information Sciences and Systems*, The Johns Hopkins University, Baltimore, pp. 91–95.
4. E. M. GURARI AND I. H. SUDBOROUGH, Cutwidth problems in graphs, in “*Proceedings 19th Annual Allerton Conference on Communication, Control, and Computing*, 1981,” pp. 752–761.
5. T. LENGAUER, Upper and lower bounds on the complexity of the Min-Cut Linear Arrangement problem on trees, *SIAM J. Algebraic Discrete Methods*, (1982).
6. W. LOUI AND A. B. SHERMAN, Comparative analysis of the Cuthill–McKee ordering algorithms for sparse matrices, *SIAM J. Numer. Anal.* (1976).
7. F. MAKEDON, C. H. PAPADIMITRIOU, AND I. H. SUDBOROUGH, Topological bandwidth, in “*Proceedings 8th Colloquium on Trees in Algebra and Programming*,” in “*Lecture Notes in Computer Science*,” Vol. 159, pp. 317–331, Springer Verlag, Berlin, 1983.
8. F. MAKEDON AND I. H. SUDBOROUGH, Minimizing width in linear layouts, in “*Proceedings of International Conference on Automata, Languages, and Programming (ICALP)*,” “*Lecture Notes in Computer Science*,” Vol. 154, Springer-Verlag, Berlin, 1983.
9. B. MONIEN AND I. H. SUDBOROUGH, Bandwidth problems in graphs, in “*Proceedings 18th Annual Allerton Conference on Communication, Control, and Computing*, 1980,” pp. 650–659.
10. C. H. PAPADIMITRIOU, The NP-completeness of the bandwidth minimization problem, *Computing* **16** (1976), 237–267.
11. W. J. SAVITCH, Relationships between nondeterministic and deterministic tape complexities, *J. Comput. System Sci.* **4** (1970), 177–192.
12. J. B. SAXE, Dynamic-programming algorithms for recognizing small bandwidth graphs in polynomial time, *SIAM J. Algebraic Discrete Methods* (1980). Vol. no. unknown.
13. L. STOCKMEYER, private communication to M. Garey and D. S. Johnson, cited in “*Computers and Intractability: A Guide to the Theory of NP-Completeness*,” Freeman, San Francisco, 1979.
14. S. TRIMBERG, Automating chip layout, *IEEE Spectrum* (1982), 38–45.
15. A. WEINBERGER, Large scale integration of MOS complex logic: A layout method, *IEEE J. Solid State Circuits* **SC-2** (1967), 182–190.
16. M. YANNAKAKIS, A polynomial algorithm for the min cut linear arrangement of trees, in “*Proceedings 24th Annual IEEE Foundations of Computer Science Symposium*, 1983,” pp. 274–281.